Reviewer Pack — Completely Bounded Norm Gap in Read-Twice BPs for IP_2

Entry e1525ec3333f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 19:34:08 UTC

Completely Bounded Norm Gap in Read-Twice BPs for IP_2

Entry ID: `e1525ec3333f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 19:34:08 UTC

1. Conjecture

Field A (mathematical branch): Operator Space Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> Let M be the transition matrix of a read-twice BP P. The completely bounded norm ||M||*{cb} satisfies* ||*M*||{cb} = O(log size(P)) for all P, but ||M||_{cb} = Ω(n) for any BP computing IP_2 mod 2.

Rationale (proposer's reasoning):

> Operator space norms capture non-commutative structure of matrices, which may expose inherent complexity in read-twice BPs. IP_2's exponential lower bound for read-once BPs suggests its read-twice representation must exhibit nontrivial operator space properties.

Taxonomy category: `BP_READTWICE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `92ce72f79ed4c10a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        # Generate a read-twice BP for IP_2 mod 2
        A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        B = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        return A, B

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def transpose(matrix):
        n = len(matrix)
        T = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                T[j][i] = matrix[i][j]
        return T

    def is_positive_definite(matrix):
        n = len(matrix)
        A = [row[:] for row in matrix]
        for k in range(n):
            pivot = A[k][k]
            if pivot <= 0:
                return False
            for j in range(k, n):
                A[k][j] /= pivot
            for i in range(k + 1, n):
                factor = A[i][k]
                for j in range(k, n):
                    A[i][j] -= factor * A[k][j]
        return True
```

```
def cholesky_decomposition(matrix):
    n = len(matrix)
    L = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1):
            sum_k = sum(L[i][k] * L[j][k] for k in range(j))
            if i == j:
                L[i][j] = math.sqrt(matrix[i][i] - sum_k)
            else:
                L[i][j] = (matrix[i][j] - sum_k) / L[j][j]
    return L

def solve_linear_system(L, b):
    n = len(L)
    y = [0] * n
    for i in range(n):
        sum_j = sum(L[i][j] * y[j] for j in range(i))
        y[i] = (b[i] - sum_j) / L[i][i]

    x = [0] * n
    for i in range(n - 1, -1, -1):
        sum_j = sum(L[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (y[i] - sum_j) / L[i][i]
    return x

def matrix_inverse(matrix):
    n = len(matrix)
    A = [row[:] for row in matrix]
    I = [[Fraction(0) if i != j else Fraction(1) for j in range(n)] for i in range(n)]

    for k in range(n):
        pivot = A[k][k]
        for j in range(k, n):
            A[k][j] /= pivot
        for j in range(n):
            I[k][j] /= pivot

        for i in range(n):
            if i != k:
                factor = A[i][k]
                for j in range(k, n):
                    A[i][j] -= factor * A[k][j]
                for j in range(n):
                    I[i][j] -= factor * I[k][j]

    return I

def spectral_norm(matrix):
    n = len(matrix)
    identity = [[Fraction(0) if i != j else Fraction(1) for j in range(n)] for i in range(n)]
    A = [row[:] for row in matrix]
    B = identity

    for _ in range(50):  # Power iteration
        B = matrix_multiplication(A, B)
        norm_B = math.sqrt(sum(sum(x**2 for x in row) for row in B))
```

```
            B = [[x / norm_B for x in row] for row in B]

        return max(abs(eigenvalue) for eigenvalue in solve_linear_system(matrix_inverse(B), [sum(r

    n = random.randint(5, 40)
    A, B = generate_read_twice_bp(n)

    M = matrix_multiplication(A, transpose(B))

    cb_norm = spectral_norm(M)

    return {
        "metric_name": "cb_norm",
        "metric_value": cb_norm,
        "instances_tested": 1,
        "conjecture_holds": cb_norm >= n / 20,  # c=1/20 as a simple lower bound
        "counterexample": "" if cb_norm >= n / 20 else f"cb_norm={cb_norm} < {n/20}"
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [1009, 1013, 1019, 1021, 1031, 1033, 1039, 1049, 1051,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_cb_norm = sum(res["metric_value"] for res in results) / len(results)
    std_cb_norm = math.sqrt(sum((res["metric_value"] - mean_cb_norm) ** 2 for res in results) / le
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_cb_norm} std={std_cb_norm} support_fraction={support_
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"{res['counterexample']}\" first_failing_seed={f
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'cb_norm', 'metric_value': 4.031266373900718e-25, 'instances_tested': 1, 'con
25 < 1.65'}
TRIAL: {'metric_name': 'cb_norm', 'metric_value': 5.271483498975786e-26, 'instances_tested': 1, 'con
26 < 1.15'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ab3931c.py", line 140, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ab3931c.py", line 124, in run_trial
    cb_norm = spectral_norm(M)
              ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ab3931c.py", line 117, in spectral_nor
    return max(abs(eigenvalue) for eigenvalue in solve_linear_system(matrix_inverse(B), [sum(row[i] fo
                                                                                    ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ab3931c.py", line 92, in matrix_invers
    A[k][j] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with ZeroDivisionError, invalidating all trial results. The counterexamples shown may be artifacts of failed computation. | next: Fix the matrix inversion division-by-zero error in spectral_norm implementation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	78647
2	preregistration	ollama_remote	qwen3:8b	0	0	23966
3	novelty	ollama_remote	qwen3:8b	0	0	20545
4	novelty	ollama_remote	qwen3:8b	0	0	19084
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42776
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17738
7	judge	ollama_remote	qwen3:8b	0	0	17995

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 220751 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/e1525ec3333f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e1525ec3333f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e1525ec3333f.tar.gz` (if generated)